



Departamento de Informática

Bacharelado em Ciência da Computação

CI1212 Arquitetura de Computadores

coprocessador aritmético 8087

adaptado de apresentação de Frederick Neal

PE01-2020
Prof. Eduardo Todt
Versão 202007111547



Objetivos

Conhecer

conceito de coprocessador aritmético

interligação com processador

aspecto histórico do tema

Intel 8087

Na início da década de 1980, a Intel lançou o 8087, primeiro coprocessador aritmético para seus processadores 16 bits





Por que coprocessador?

Processadores convencionais na época, como 8086 ou 8088, realizavam funções como log, seno e outras com bibliotecas matemáticas que demandavam muita carga computacional

Com o coprocessador, estas operações passaram a serem realizadas em hardware

Coprocessador porque não cabia tudo de forma viável em um único circuito integrado

Aumento em desempenho

Execução com o coprocessador 8087 vs emulação no 8086 por meio de bibliotecas

Repare na frequência de clock do processador ...

Instruction	Approximate Execution Time (μs) (5-MHz clock)	
	8087	8086 Emulation
Multiply (single precision)	19	1,600
Multiply (double precision)	27	2,100
Add	17	1,600
Divide (single precision)	39	3,200
Compare	9	1,300
Load (single precision)	9	1,700
Store (single precision)	18	1,200
Square root	36	19,600
Tangent	90	13,000
Exponentiation	100	17,100

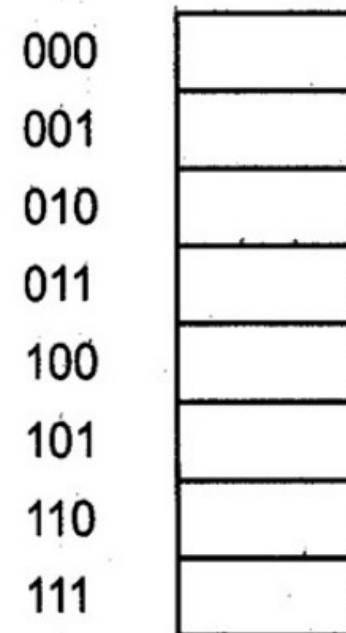
Registadores

8 registradores de
80 bits

Acessados como
pilha (conhece
cálculo com RPN? Se
não, pesquise)

ST(0) é o registrador
topo da pilha

8087 registers



Exemplo

```

;Program for Example 20-5 to load 3 numbers and compute their sum
.8087
PAGE 60,132
STACKSG SEGMENT
        DW 32 DUP (?)
STACKSG ENDS

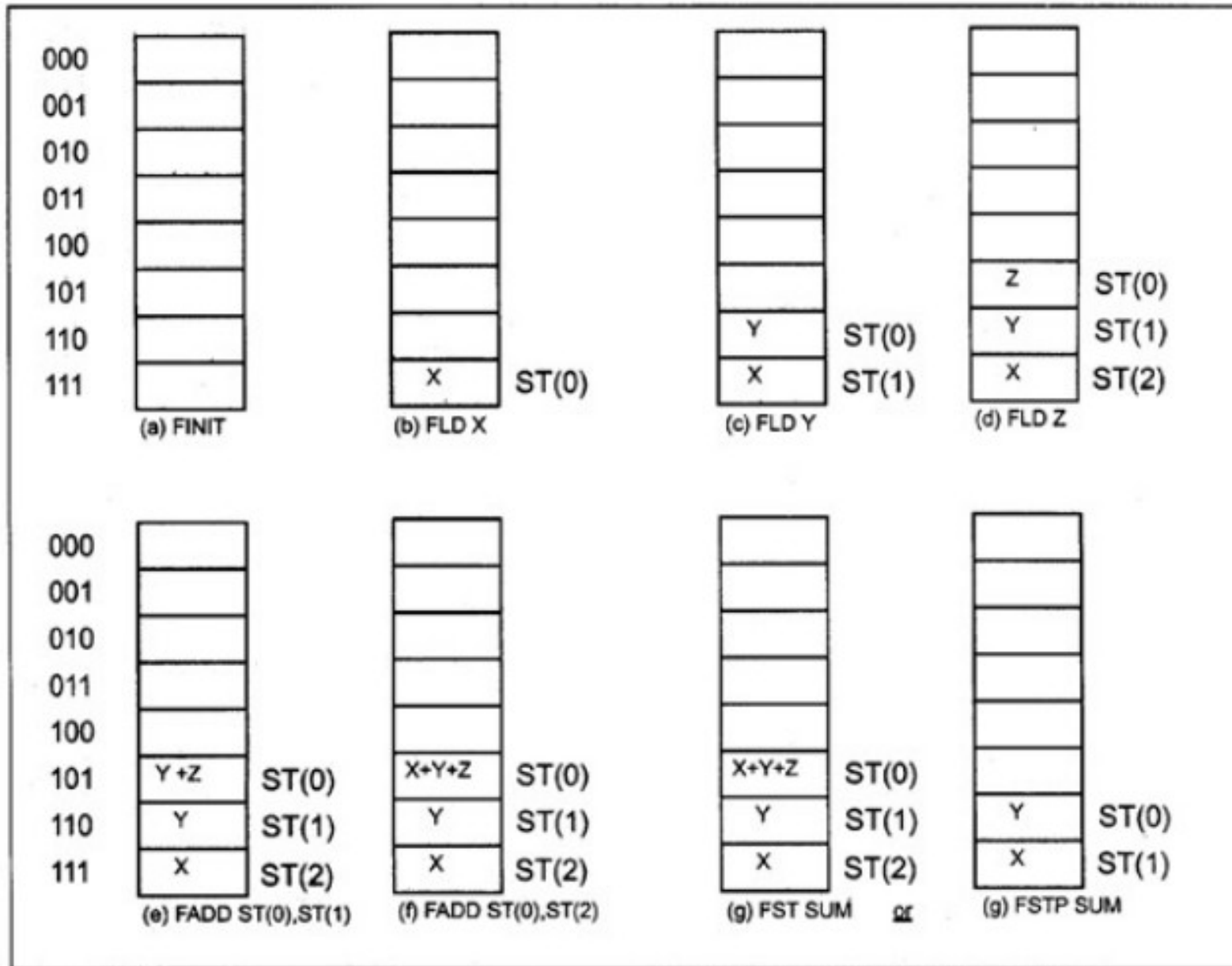
;-----
DATASG SEGMENT
        ORG 00H
X        DD 9.75
        ORG 10H
Y        DD 13.09375
        ORG 20H
Z        DD 29.0390625
        ORG 30H
SUM       DD ?
DATASG ENDS

;-----
CODESG SEGMENT
START    PROC FAR
        ASSUME CS:CODESG,DS:DATASG,SS:STACKSG
        MOV AX,DATASG
        MOV DS,AX
        CALL CSUM
        MOV AH,4CH
        INT 21H
START    ENDP

;-----
CSUM      PROC NEAR
        FINIT                                ;initialize 8087 stack
        FLD X                                ;load X into ST(0)
        FLD Y                                ;load Y into ST(0)
        FLD Z                                ;load Z into ST(0)
        FADD ST(0),ST(1)                     ;ST(0) = Y + Z
        FADD ST(0),ST(2)                     ;ST(0) = X + (Y + Z)
        FST SUM                               ;store ST(0) in sum
        RET
CSUM      ENDP
CODESG    ENDS
END        START

```

Pilha para este exemplo





IEEE 754

Até final da década de 1970 cada fabricante representava float do seu jeito

Problemas sérios de compatibilidade

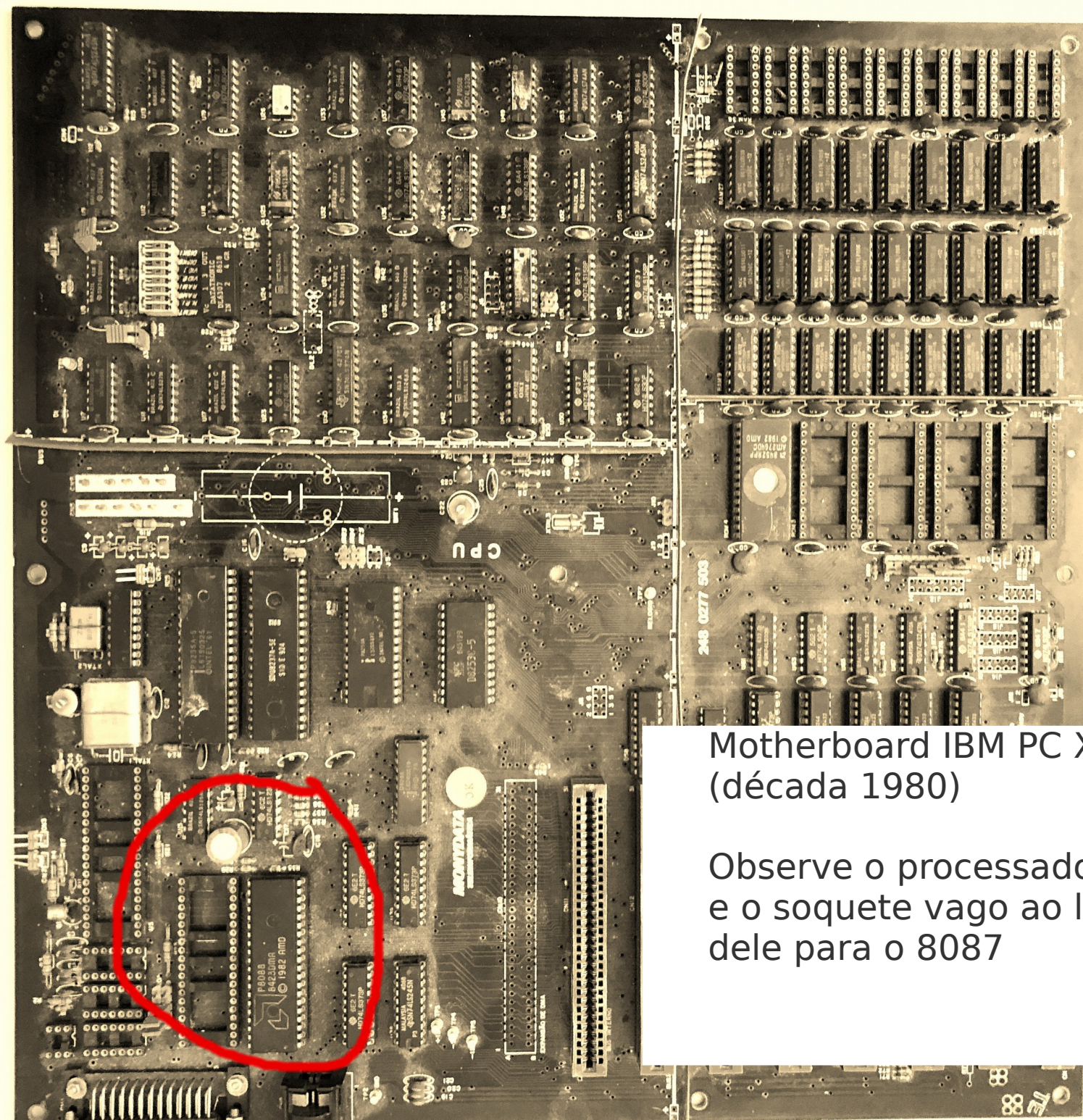
Em 1980 IEEE normatizou a representação float com a IEEE 754, com forte influência da Intel, baseada no 8087



Execução

Durante a execução pode ser detectada a presença do coprocessador

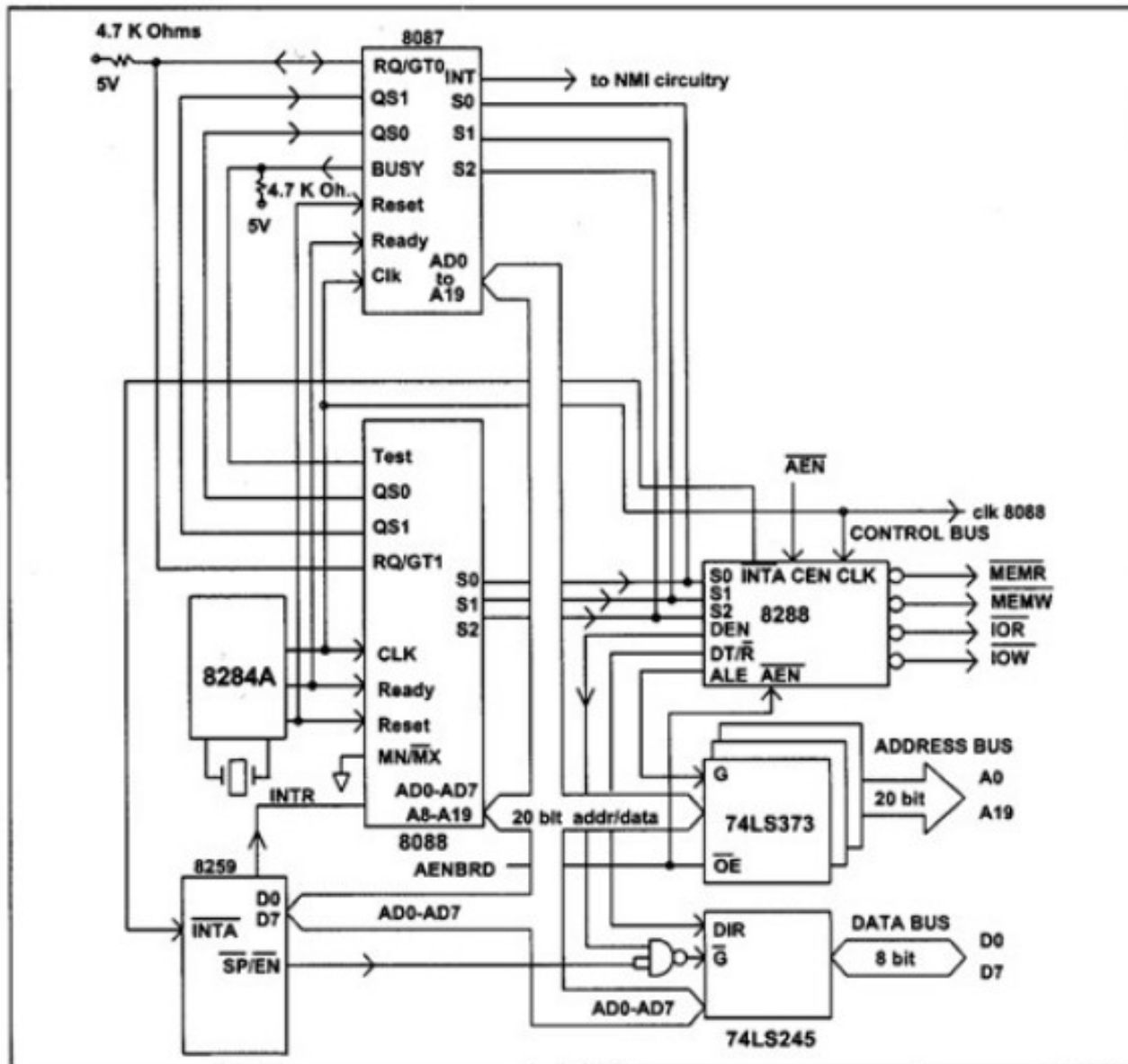
Se estiver presente a execução é com ele, caso contrário via bibliotecas de emulação



Motherboard IBM PC XT
(década 1980)

Observe o processador 8080
e o soquete vago ao lado
dele para o 8087

Conexão 8088 - 8087





Funcionamento

Muito legal: o 8087 fica observando instruções passarem no barramento. Se for um que ele executa, então a considera.

Sinais entre eles ajudam na comunicação

Sinais de interação

1. The 8088 and 8087 receive the same signals, CLK, READY, and RESET, from the 8284. This ensures that they are synchronized.
2. S0, S1, and S2 are going from the 8088 or 8087 to the 8288, which allows either of these two processors to provide the status signal to the 8288.
3. The Queue Status, QS1 and QS2, from the 8088 go to the 8087, allowing it to know the status of the queue of the 8088 at any given time.
4. The TEST signal to the 8088 comes from BUSY of the 8087.
By deactivating (going low) the BUSY signal, the 8087 informs the 8088 that it finished execution of the instruction which it has been WAITing for.
5. RQ/GT1 (request/grant) of the 8088 is connected to RQ/GTO of the 8087, allowing them to arbitrate mastery over the buses.
There are two sets of RQ/GT: RQ/GT1 and RQ/GTO . RQ/GT1 of the 8087 is not used and is connected to Vcc permanently.
This extra RQ/GT is provided in case there is a third microprocessor connected to the local bus.

Sinais de interação

6. Both the 8088 and 8087 share buses ADO -AD7 and A8 -A19, allowing either one to access memory.
Since the 8087 is designed for both the 8088 and 8086, signal BHE is provided for the 8086 processor.
It is connected to Vcc if the 8087 is used with the 8088.
If the microprocessor used was an 8086, BHE from the 8086 is connected to BHE of the 8087.
7. INT of the 8087 is an **output signal indicating error conditions**, also called exceptions, **such as divide by zero**. Error conditions are given in the status word. Assuming the bit for that error is not masked and an interrupt is enabled, whenever any of these errors occurs, the 8087 automatically activates the INT pin by putting high on it. In the IBM PC and compatibles, this signal is connected to the NMI circuitry as discussed in Chapter 14.
Since there is only one INT for all error conditions (exceptions) of the 8087, it is up to the programmer to write a program to **check the status word** to see which has caused the error.
8. The 8088, often called the **host processor, must be connected in maximum mode** to be able to accommodate a coprocessor such as the 8087.

Operação conjunta

- This is especially critical since the 8088 can fetch memory in 4 clocks, while some 8087 instructions take over 100 clocks to execute.
- It is the job of the assembler to put a WAIT instruction between two consecutive 8087 instructions.
- Verify this by looking at the EXE file of any previous example.
- In addition, when the 8087 is executing an instruction, it activates the BUSY pin automatically by putting high on it.
- This pin is connected to the TEST pin of the 8088/86.
- Next, the 8088/86 fetches the next instruction, which is a WAIT instruction that has been inserted by the assembler, and executes it, thereby going into an internal loop while continuously monitoring the TEST input pin to see when this pin goes low.
- When the 8087 finishes execution of the present instruction, it pulls down (low) the READY pin, indicating through the TEST pin to the 8088/86 that it can now send the next instruction to the 8087.

Cooperando

- Each gets a copy of the instructions as they are fetched from memory.
- Since all the instructions of the 8087 have 9BH in the most significant byte of the opcode, the 8088/86 ignores these instructions.
- In reality, 9BH is the opcode for the 8088/86 ESCAPE instruction.
- Likewise, the 8087 ignores any opcode that lacks 9BH.
- It must be made clear that although both receive a copy of each fetched opcode, only the 8088/86 can fetch opcodes since it is the only device that has the instruction pointer.
- Now one might ask how the 8088/86 makes sure it is not flooding the 8087 by fetching instructions for the coprocessor faster than the 8087 can process them.
- The first rule of working together is that the 8088/86 cannot fetch another 8087 instruction until the 8087 has finished execution of the present instruction.

Cooperando

- This is especially critical since the 8088 can fetch memory in 4 clocks, while some 8087 instructions take over 100 clocks to execute.
- It is the job of the assembler to **put a WAIT instruction** between two consecutive 8087 instructions.
- Verify this by looking at the EXE file of any previous example.
- In addition, **when the 8087 is executing an instruction, it activates the BUSY pin automatically by putting high on it.**
- This pin is connected to the TEST pin of the 8088/86.
- Next, the 8088/86 fetches the next instruction, which is a WAIT instruction that has been inserted by the assembler, and executes it, thereby going into an internal loop while **continuously monitoring the TEST input pin** to see when this pin goes low.
- When the 8087 finishes execution of the present instruction, it pulls down (low) the READY pin, indicating through the TEST pin to the 8088/86 that it can now send the next instruction to the 8087.

- 
- Although the 8088/86 fetches opcodes for the 8087, **it cannot fetch operands for the 8087** because the 8088/86 has no knowledge of the nature of 8087 operands **since it does not execute 8087 instructions**.
 - So the question is: How are 8087 operands accessed from memory?
 - For the sake of clarity, read and write cases will be looked at separately.

Lendo operando

- Assume that the 8087 needs to read an operand. When the 8088/86 initiates the operand read cycle, the 8087 grabs the 20-bit address and saves it internally.
- If the operand is a single word (like a word integer), the read cycle has been initiated and the word will come into both processors.
- Only the 8087 will use the data; the 8088/86 will ignore it. However, if the operand is 32 bits or longer, the 8087 will take over the buses by sending a low pulse on its RQ/GTO to the RQ/GT1 of the 8088/86 (see Figure 20-4).

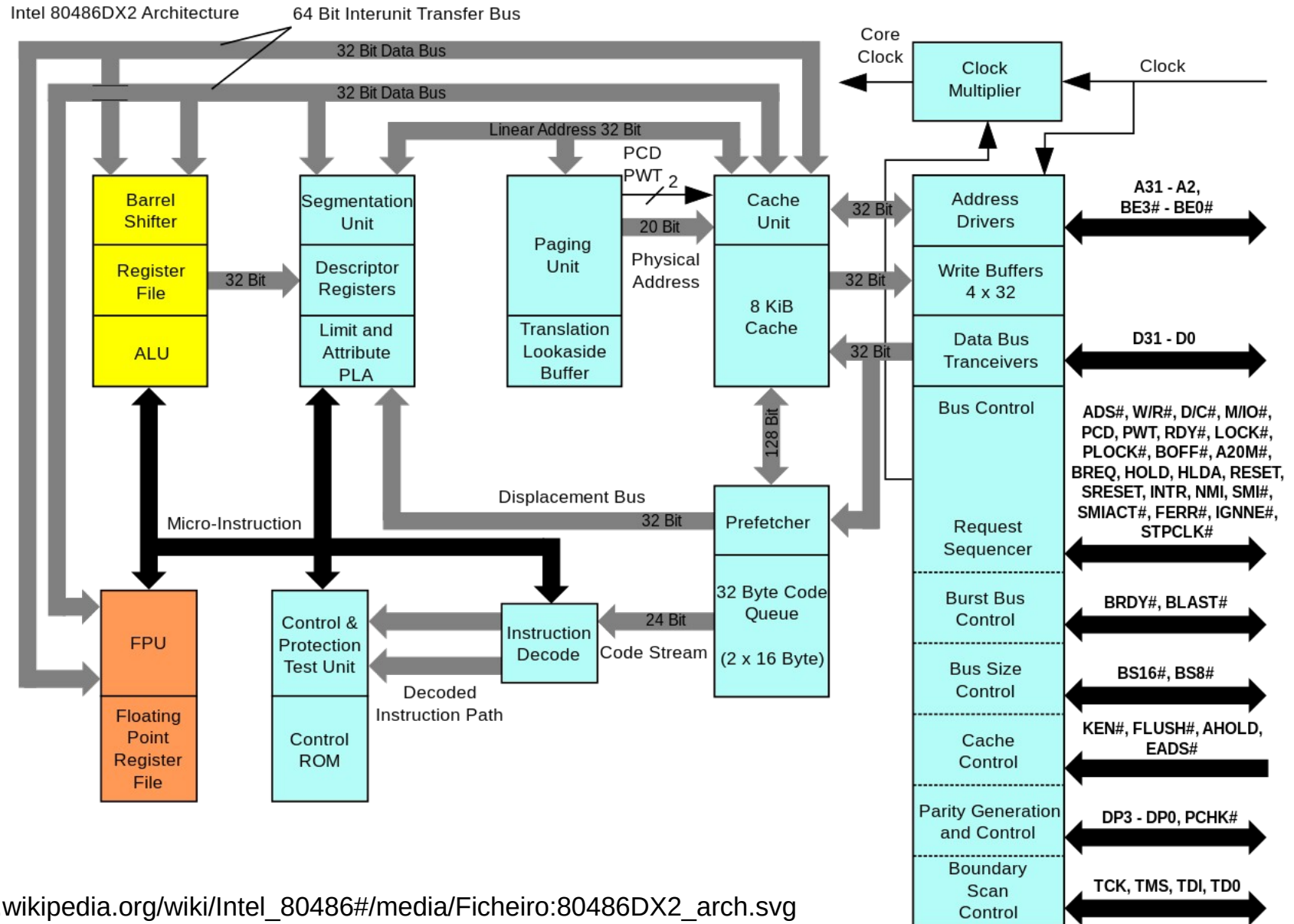
- 
- The 8088/86 in turn will send back a low pulse through the same pin, thereby allowing the 8087 to take over the buses.
 - Remember, RQ/GT is a **bidirectional** bus.
 - When the 8087 takes over the buses, it will use them until it brings in the last byte of the operand.
 - It is only then that by activating RQ/GT (making it low), control of the buses is given back to the 8088/86.
 - For example, in the case of a DT operand, the 8087 has control over the buses for the time needed to fetch all 10 bytes and then it gives back the buses.

Escrevendo um operando

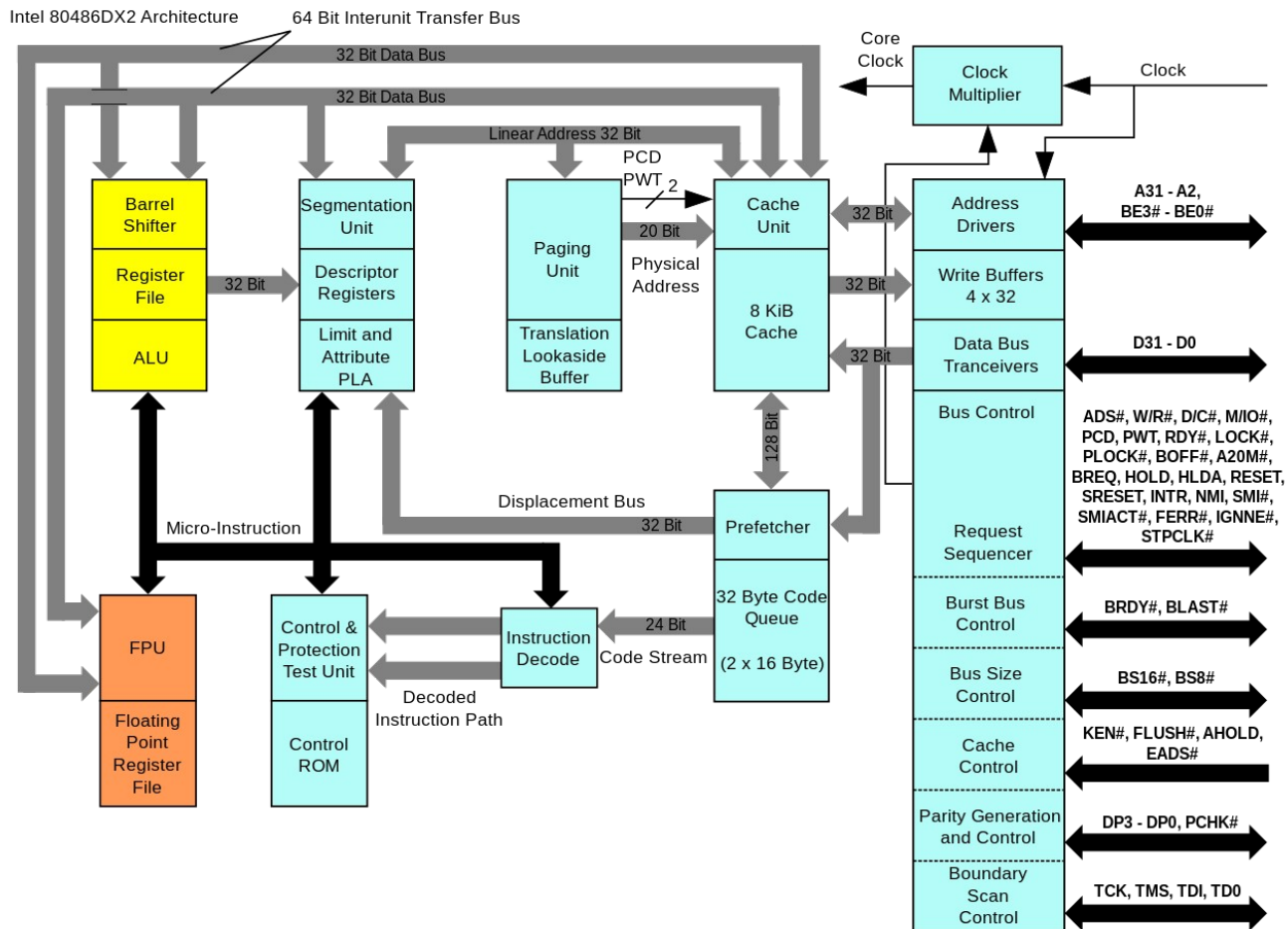
- In the case of writing an operand by the coprocessor (e.g., FST data), the 8088/86 initiates the write cycle, but the 8087 ignores it since the 8086 does not have the operand.
- This is called a *dummy cycle*.
- All the 8087 does **during the dummy cycle is grab the address of the first memory location** where the operand is to be stored and keep it until the data is ready, and then it requests the use of the buses by activating the RQ/GT pin.
- From then on, the process is the same as the read cycle, meaning that it will use the buses until it writes the last byte of the operand.
- All the cases discussed so far have been taken care of by either the assembler or the hardware and there was no need for the programmer to be worried.

- 
- However, One case where the programmer must get involved is when the result of an operation performed by the 8087 needs to be used by the 8088/86.
 - The programmer must ensure that this result is available before the 8088/86 uses it.
 - This is done by putting an FWAIT instruction in between the 8087 instruction and the subsequent 8088/86 instruction, which needs to use the result.

Em 1989 o Intel 486 !

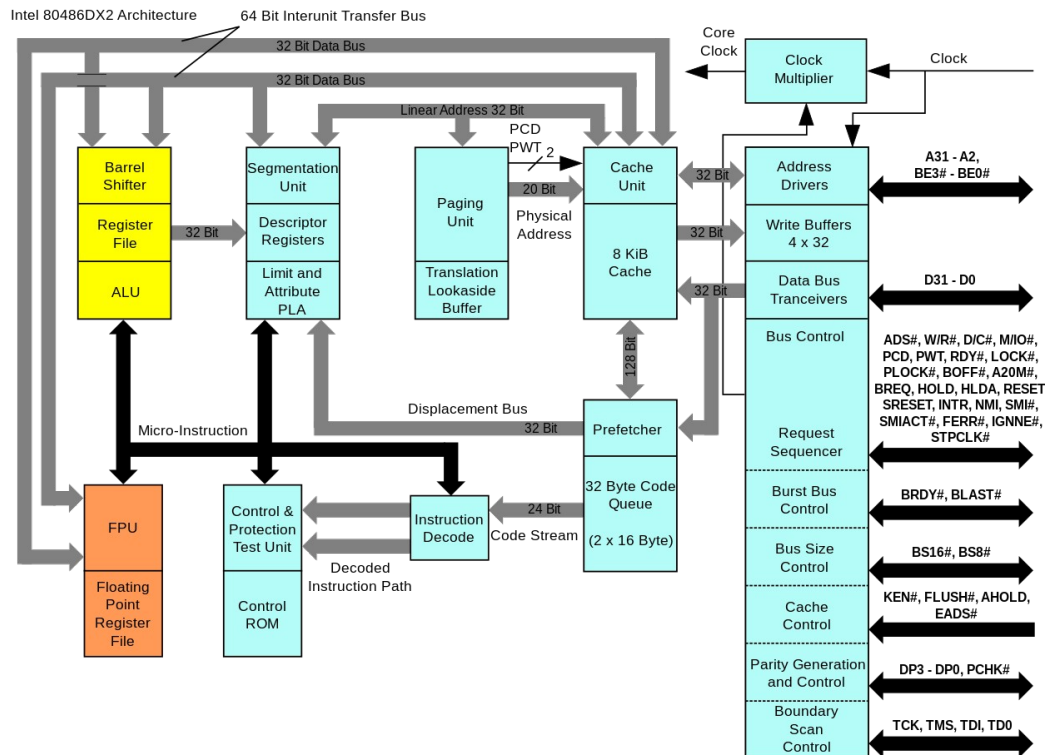


Em 1989 o Intel 486 !



Aqui já foi possível integrar o coprocessador aritmético no mesmo circuito integrado (FPU), acabando com a sequência dos 8087, 8027 e 80387.

Em 1989 o Intel 486 !



Curiosidade: existia versão do processador sem FPU, o i486SX.

Não é que a Intel sempre quisesse vender isso, mas não conseguia fabricar com todos funcionando.

Depois esta opção também foi forma de ocupar nicho de mercado concorrendo com a AMD, que clonava seus processadores, fabricando versões com menor desempenho.

Os que não funcionavam tinham a FPU desabilitada por corte laser e eram vendidos como processadores mais baratos.



Referência

The 80x87 Math Coprocessor. Why we need a math coprocessor? Using a general-purpose microprocessor such as the 8088/86 to perform mathematical functions.

Autor: Frederick Neal

CI1212

Arquitetura de Computadores



Prof. Eduardo Todt

todt@inf.ufpr.br

Período Especial 2020 01

